Due Date: 3/12/2026 Names: Aiden Wang

CSCI 240 PA2 Submission

Exercise 1

Source code below:

```
// Modified by: Aiden Wang
package PA2;

import java.util.Random;

public class PA2_EX1 {

   // O(n^2) Algorithm
   public static double[] prefixAverage1(double[] x) {
       int n = x.length;
       double[] a = new double[n];
       for (int j = 0; j < n; j++) {
           double total = 0;
           for (int i = 0; i <= j; i++) {
               total += x[i];
           }
           a[j] = total / (j + 1);
       }
       return a;
   }

   // O(n) Algorithm
   public static double[] prefixAverage2(double[] x) {
       int n = x.length;
       double[] a = new double[n];
       double total = 0;
       for (int j = 0; j < n; j++) {
           total += x[j];
           a[j] = total / (j + 1);
       }
       return a;
   }

   public static void main(String[] args) {
       System.out.println("Author: Aiden Wang\n");

       // 1. Test case to confirm correctness
       double[] testArr = {31, -41, 59, 26, -53, 58, 97, -93, -23, 84};
       double[] result = prefixAverage2(testArr);
       System.out.print("Prefix averages of test case: ");
       for (double val : result) {
           System.out.printf("%.2f ", val);
       }
       System.out.println("\n");

       // 2. Running Time Experiments
       int[] sizes = {100, 1000, 10000, 100000};
       Random rand = new Random();

       System.out.printf("%-10s | %-20s | %-20s\n", "n", "prefixAverage1
(ns)", "prefixAverage2 (ns)");
```

```
System.out.println("-------------------------------------------------------
----------");

        for (int n : sizes) {
            double[] x = new double[n];
            for (int i = 0; i < n; i++) {
                x[i] = -n + (rand.nextDouble() * (2 * n)); // Random
between -n and n
            }

            long startTime1 = System.nanoTime();
            prefixAverage1(x);
            long endTime1 = System.nanoTime();
            long time1 = endTime1 - startTime1;

            long startTime2 = System.nanoTime();
            prefixAverage2(x);
            long endTime2 = System.nanoTime();
            long time2 = endTime2 - startTime2;

            System.out.printf("%-10d | %-20d | %-20d\n", n, time1, time2);
        }

        System.out.println("\nConclusion: The times collected match the
Big-O notation. prefixAverage1 grows quadratically (O(n^2)), while
prefixAverage2 grows linearly (O(n)).");
    }
}
```

Input/output below:

```
Author: Aiden Wang

Prefix averages of test case: 31.00 -5.00 16.33 18.75 4.40 13.33 25.29 10.50 6.78 14.50

n         | prefixAverage1 (ns)  | prefixAverage2 (ns)
-------------------------------------------------------------
100       | 42916              | 1916
1000      | 1714000            | 15958
10000     | 53276458           | 160541
100000    | 4809062166         | 1497916

Conclusion: The times collected match the Big-O notation. prefixAverage1 grows quadratically (O(n^2)), while prefixAverage2 grows linearly (O(n)).
```

Exercise 2

Due Date: 3/12/2026 Names: Aiden Wang

Source code below:

```java
// Modified by: Aiden Wang
package PA2;
class PA2_Ex2 {
   static int recursiveCalls = 0;
   public static void findPair(int[] A, int k, int low, int high) {
       recursiveCalls++;
       // Base case: If bounds cross, no pair exists
       if (low >= high) {
           System.out.println("Result: no pair exists");
           return;
       }
       int sum = A[low] + A[high];
       if (sum == k) {
           System.out.println("Result: values " + A[low] + " and " +
A[high]);
       } else if (sum < k) {
           findPair(A, k, low + 1, high); // Need a larger sum
       } else {
           findPair(A, k, low, high - 1); // Need a smaller sum
       }
   }
   public static void main(String[] args) {
       System.out.println("Author: Aiden Wang\n");
       int[] A = {3, 9, 12, 15, 16, 23};
       // Test Case 1
       System.out.println("Test Case: k = 25");
       recursiveCalls = 0;
       findPair(A, 25, 0, A.length - 1);
       System.out.println("Recursive calls: " + recursiveCalls + "\n");
       // Test Case 2
       System.out.println("Test Case: k = 16");
       recursiveCalls = 0;
       findPair(A, 16, 0, A.length - 1);
       System.out.println("Recursive calls: " + recursiveCalls);
   }
}
```

Input/output below:

```
/Users/gooqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Hom
Author: Aiden Wang

Test Case: k = 25
Result: values 9 and 16
Recursive calls: 3

Test Case: k = 16
Result: no pair exists
Recursive calls: 6

Process finished with exit code 0
```

Due Date: 3/12/2026 Names: Aiden Wang

Exercise 3
Source code below:

```
// Modified by: Aiden Wang
package PA2;
class PA2_Ex3 {
   // Setup global/static variables
   static String[] arr = new String[4];
   static int numElems = 0;

   public static void insertRear(String name) {
       if (numElems < 10) {
           arr[numElems] = name;
           numElems++;
       }
   }

   public static void insertAt(int index, String name) {
       if (numElems < 10 && index >= 0 && index <= numElems) {
           for (int i = numElems; i > index; i--) {
               arr[i] = arr[i - 1];
           }
           arr[index] = name;
           numElems++;
       }
   }

   public static void removeRear() {
       if (numElems > 0) {
           numElems--;
           arr[numElems] = null;
       }
   }

   public static void removeAt(int index) {
       if (index >= 0 && index < numElems) {
           for (int i = index; i < numElems - 1; i++) {
               arr[i] = arr[i + 1];
           }
           numElems--;
           arr[numElems] = null;
       }
   }

   public static void printAll() {
       for (int i = 0; i < numElems; i++) {
           System.out.print(arr[i] + " ");
       }
       System.out.println();
   }

   public static void main(String[] args) {
       System.out.println("Author: Aiden Wang\n");

       insertRear("Jo");
       insertRear("Jane");
       insertAt(1, "John");
```

```
        insertRear("Kim");

        System.out.println("numElems: " + numElems);  // output 4

        removeAt(0);
        removeRear();

        System.out.print("List: ");
        printAll();    // output John Jane
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/C
Author: Aiden Wang

numElems: 4
List: John Jane

Process finished with exit code 0
```

Exercise 4
Source code below:

```
// Modified by: Aiden Wang
package PA2;
class PA2_EX4 {
    // Setup Node
    static class Node {
        String data;
        Node next;
        Node(String data) {
            this.data = data;
            this.next = null;
        }
    }

    // Setup global/static variables
    static Node head = null;
    static int numElems = 0;

    public static void insertFront(String name) {
        Node newNode = new Node(name);
        newNode.next = head;
        head = newNode;
        numElems++;
    }

    public static void insertRear(String name) {
        Node newNode = new Node(name);
```

```
        if (head == null) {
            head = newNode;
        } else {
            Node curr = head;
            while (curr.next != null) {
                curr = curr.next;
            }
            curr.next = newNode;
        }
        numElems++;
    }

    public static void removeElem(String name) {
        if (head == null) {
            System.out.println("Unsuccessful");
            return;
        }
        // If the element to remove is at the head
        if (head.data.equals(name)) {
            head = head.next;
            numElems--;
            System.out.println("Successful");
            return;
        }
        // Search for the element
        Node curr = head;
        while (curr.next != null && !curr.next.data.equals(name)) {
            curr = curr.next;
        }
        if (curr.next != null) {
            curr.next = curr.next.next;
            numElems--;
            System.out.println("Successful");
        } else {
            System.out.println("Unsuccessful");
        }
    }

    public static void printAll() {
        Node curr = head;
        while (curr != null) {
            System.out.print(curr.data + " ");
            curr = curr.next;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        System.out.println("Author: Aiden Wang\n");

        insertFront("Jo");
        insertRear("Jane");
        insertFront("John");
        insertRear("Kim");

        System.out.println("numElems: " + numElems);  // output 4
```

```
        removeElem("Jane");  // successful
        removeElem("Bob");   // unsuccessful

        System.out.print("List: ");
        printAll();    // output John Jo Kim
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Content
Author: Aiden Wang

numElems: 4
Successful
Unsuccessful
List: John Jo Kim

Process finished with exit code 0
```

Answer for Question 1
Question: Would it be more efficient to use a tail variable for the linked list in exercise 4? Explain.
Answer:
Yes, for insertions.
Adding a tail reference improves the efficiency of insertRear from O(n) to O(1).
Without Tail: You must traverse the entire list from the head to find the last node (O(n)).
With Tail: You have a direct "shortcut" to the end, allowing an immediate link (O(1)).
(Note: However, keeping a tail variable would not speed up removing the rear element in a singly linked list, because you still have to traverse from the head to find the node immediately preceding the tail to update its next reference to null).

Answer for Question 2
Question: Perform exercise 4.5.27 in Java book; set up a table showing approximate values for requested run times for the 2 columns, second and hour.
Answer:
Assuming the algorithm takes f(n) microseconds to execute.
1 Second = 10^6 microseconds (operations)
1 Hour = 3.6×10^9 microseconds (operations)
Here is the table showing the maximum problem size n for each standard Big-O function:

Due Date: 3/12/2026 Names: Aiden Wang

Function f(n)	Maximum n for 1 Second	Maximum n for 1 Hour
$\log_2 n$	2^{10^6}	$2^{3.6\times10^9}$
$\sqrt{n}$	10^{12}	1.296×10^{19}
n	10^6	3.6×10^9
$n\log_2 n$	≈62,746	≈133,378,058
n^2	1,000	60,000
n^3	100	≈1,532
2^n	19	31
n!	9	12

Extra Credit – Option 2 (Maximum Contiguous Sub-array)

Source code below:

```java
package PA2;

// Author: Aiden Wang
import java.util.Random;

public class PA2_EC2 {

   // O(n^3) Algorithm using a triple nested loop
   public static int maxSubArraySum(int[] x) {
       int n = x.length;
       int maxSum = Integer.MIN_VALUE;

       for (int i = 0; i < n; i++) {
           for (int j = i; j < n; j++) {
               int currentSum = 0;
               for (int k = i; k <= j; k++) {
                   currentSum += x[k];
               }
               if (currentSum > maxSum) {
                   maxSum = currentSum;
               }
           }
       }
       return maxSum;
   }

   public static void main(String[] args) {
       System.out.println("Author: Aiden Wang\n");
```

Due Date: 3/12/2026 Names: Aiden Wang

```
        int[] test1 = {31, -41, 59, 26, -53, 58, 97, -93, -23, 84};
        System.out.println("Test 1 Max Sum (Expected 187): " +
maxSubArraySum(test1));

        int[] test2 = {31, 41, 59, 26, 53, 58, 97, 93, 23, 84};
        System.out.println("Test 2 Max Sum (Expected 565): " +
maxSubArraySum(test2));

        // Timing Experiments
        int[] sizes = {100, 1000, 10000};
        Random rand = new Random();

        System.out.println("\nRunning Time Experiments:");
        System.out.printf("%-10s | %-15s\n", "n", "Time (ms)");
        System.out.println("-----------------------------");

        for (int n : sizes) {
            int[] x = new int[n];
            for (int i = 0; i < n; i++) {
                x[i] = -n + rand.nextInt(2 * n + 1);
            }

            long startTime = System.currentTimeMillis();
            maxSubArraySum(x);
            long timeMs = System.currentTimeMillis() - startTime;

            System.out.printf("%-10d | %-15d\n", n, timeMs);
        }
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/
Author: Aiden Wang

Test 1 Max Sum (Expected 187): 187
Test 2 Max Sum (Expected 565): 565

Running Time Experiments:
n          | Time (ms)
-----------------------------
100        | 1
1000       | 106
10000      | 107789

Process finished with exit code 0
```